

Proyecto Final: Videojuego tipo Pac-Man en C con SDL2

Murrieta Sánchez Luis Santiago
Arvizu Javier Moisés
Martinez Arellano Elizabeth Yael
Fundamentos de Programación
Universidad Nacional Autónoma de México

Semestre 2026-2

Índice

1. Introducción	3
2. Objetivo general	3
3. Objetivos específicos	3
4. Descripción general del juego	3
5. Requisitos funcionales	4
5.1. Ventana gráfica	4
5.2. Mapa del juego	4
5.3. Carga de mapa desde archivo	4
5.4. Movimiento del jugador	5
5.5. Fantasmas	5
5.6. Sistema de puntaje y vidas	5
5.7. Fin del juego	5
5.8. Archivos	6
5.9. Sprites y apartado visual	6

6. Requisitos técnicos	6
7. Análisis del problema	6
7.1. Entradas	6
7.2. Procesos	7
7.3. Salidas	7
8. Diagramas de flujo	7
8.1. Ciclo principal	7
8.2. Movimiento	7
8.3. Colisiones	7
8.4. Fantasmas	7
8.5. Control del juego	7
9. Pseudocódigo	12
9.1. Módulo cargarMapa	12
9.2. Módulo detectarColisiones	12
9.3. Módulo moverJugador	13
9.4. Módulo moverFantasma	13
9.5. Módulo guardarPuntaje	14
10.Modularización	14
11.Uso de herramientas de inteligencia artificial	14
12.Bitácora de desarrollo	15
13.Conclusiones	15

1. Introducción

El presente documento describe el desarrollo de un videojuego tipo Pac-Man simplificado, implementado en lenguaje C y con apoyo de la biblioteca SDL2. El proyecto integra los temas principales del curso de Fundamentos de Programación: resolución de problemas, diagramas de flujo, pseudocódigo, funciones, arreglos bidimensionales, manejo de archivos, modularización y programación estructurada.

La mecánica principal consiste en controlar a un personaje dentro de un laberinto, consumir pellets, evitar fantasmas y acumular puntos. El juego finaliza cuando el jugador pierde todas sus vidas o cuando se consumen todos los pellets del nivel.

2. Objetivo general

Desarrollar un videojuego tipo Pac-Man simplificado en lenguaje C utilizando SDL2, aplicando programación estructurada y los conceptos fundamentales revisados durante el curso.

3. Objetivos específicos

- Diseñar la solución mediante análisis del problema, diagramas de flujo y pseudocódigo.
- Implementar funciones para organizar el programa en módulos claros.
- Utilizar un arreglo bidimensional para representar el mapa del juego.
- Cargar el nivel desde un archivo de texto.
- Guardar el puntaje final en un archivo externo.
- Implementar movimiento del jugador, colisiones, pellets, vidas y fantasmas con reglas simples.
- Usar SDL2 para crear una ventana, renderizar elementos gráficos y capturar eventos del teclado.
- Documentar el uso responsable de herramientas de inteligencia artificial como apoyo al desarrollo.

4. Descripción general del juego

El videojuego se desarrolla en una ventana gráfica generada con SDL2. El escenario es un laberinto cargado desde el archivo `mapa.txt`. El jugador controla a Pac-Man con las flechas del teclado y debe recorrer caminos libres para consumir pellets. Cada pellet suma puntos y reduce el contador de pellets restantes.

El fantasma se desplaza mediante reglas simples: en la mayoría de los turnos intenta acercarse al jugador comparando coordenadas, y en otros casos puede seleccionar una dirección aleatoria. Si el fantasma colisiona con el jugador, se resta una vida. La partida termina con victoria si no quedan pellets o con derrota si las vidas llegan a cero.

5. Requisitos funcionales

5.1. Ventana gráfica

El programa utiliza SDL2 para crear una ventana, dibujar el mapa, representar al jugador y al fantasma, y capturar eventos del teclado. La ventana se dimensiona a partir del número de filas, columnas y tamaño de celda:

```
#define FILAS 20
#define COLUMNAS 20
#define TAMANIO_CELDA 24
```

5.2. Mapa del juego

El mapa se representa mediante un arreglo bidimensional de caracteres:

```
char mapa[FILAS][COLUMNAS];
```

Símbolo	Significado
#	Pared
.	Pellet
P	Posición inicial del jugador
F	Posición inicial del fantasma
□	Camino libre

5.3. Carga de mapa desde archivo

El nivel se carga desde `mapa.txt`. Durante la lectura se guardan las paredes y pellets en la matriz, se localiza la posición inicial del jugador y se localiza la posición inicial del fantasma.

```
#####
#P.....#.....##
#.#####.#
#.#...#.#.#...#.#
#.#.#.#.#.###.#.#
#...#...#...#...#
#####.#####.#####
#...#...#...#...#
```

```

#.#.###.#.#.###.#.#
#.#.....#.#.....#.#
#.#####.#####.#
#.....#
#.#####.###.#####.#
#.#...#.#.#.#...#.#
#.#.#.#.#.#.#.#.#
#...#...#...#...#.#
#####.#####.#####
#F.....#.....#
#####...#####
#####

```

5.4. Movimiento del jugador

El jugador se mueve con las flechas direccionales. Para cada movimiento se calcula una nueva posición y se valida que no salga de los límites ni atravesie paredes. Si la nueva celda contiene un pellet, el puntaje aumenta y la celda queda libre.

5.5. Fantasmas

El fantasma implementa un comportamiento básico, sin inteligencia artificial avanzada ni algoritmos complejos de búsqueda. La regla principal consiste en comparar la posición del jugador con la del fantasma para intentar acercarse. También se incorpora movimiento aleatorio para evitar un comportamiento completamente predecible.

5.6. Sistema de puntaje y vidas

Cada pellet consumido suma 10 puntos. El jugador inicia con 3 vidas. Al colisionar con el fantasma, pierde una vida y las posiciones se reinician para continuar la partida si aún quedan vidas.

5.7. Fin del juego

El juego muestra el resultado final mediante mensajes de consola:

- **Victoria:** cuando `total_pellets <= 0`.
- **Derrota:** cuando `vidas <= 0`.
- **Puntaje final:** se guarda en `puntajes.txt`.

5.8. Archivos

- `mapa.txt`: archivo de entrada para cargar el nivel.
- `puntajes.txt`: archivo de salida para registrar el puntaje final.

5.9. Sprites y apartado visual

La versión propuesta usa figuras geométricas simples renderizadas con SDL2:

- Paredes: rectángulos azules.
- Pellets: cuadrados blancos pequeños.
- Jugador: bloque amarillo.
- Fantasma: bloque rojo.
- Fondo: color negro.

No se emplean sprites animados, ya que el requisito permite una representación visual simple siempre que se mantenga la mecánica principal.

6. Requisitos técnicos

Elemento	Implementación
Lenguaje	C
Biblioteca gráfica	SDL2
Paradigma	Programación estructurada
Estructura principal	Funciones y arreglos bidimensionales
Archivos	Lectura de mapa y escritura de puntajes

El proyecto no utiliza C++, motores de videojuegos, bibliotecas gráficas adicionales ni inteligencia artificial avanzada.

7. Análisis del problema

7.1. Entradas

- **Archivo de configuración del nivel:** archivo `mapa.txt` con la estructura inicial del laberinto. Los caracteres representan paredes, pellets, jugador y fantasma.
- **Eventos de teclado:** señales capturadas por SDL2 cuando el usuario presiona las flechas direccionales o la tecla `Esc`.

7.2. Procesos

- **Carga y mapeo de datos:** lectura de `mapa.txt` para copiar los caracteres al arreglo bidimensional `mapa`.
- **Cálculo de movimiento del jugador:** obtención de nuevas coordenadas a partir de la dirección indicada por el teclado.
- **Detección de colisiones:** validación de límites del mapa y bloqueo de movimientos hacia paredes.
- **Recolección y puntaje:** detección de pellets, incremento de puntaje y decremento de `total_pellets`.
- **Movimiento del fantasma:** comparación de coordenadas para perseguir parcialmente al jugador y selección aleatoria cuando corresponde.
- **Evaluación del estado del juego:** verificación de vidas, pellets restantes, victoria o derrota.

7.3. Salidas

- **Interfaz gráfica:** representación visual del laberinto, pellets, jugador y fantasma mediante SDL2.
- **Mensajes de estado:** avisos de inicio de partida, pellets consumidos, pérdida de vidas, victoria o derrota.
- **Persistencia:** escritura del puntaje final en `puntajes.txt`.

8. Diagramas de flujo

Los diagramas de flujo se tomaron directamente del documento `Diagramas de flujo.docx`. El archivo contiene cinco imágenes internas, integradas a continuación como evidencia de los procesos principales solicitados.

8.1. Ciclo principal

8.2. Movimiento

8.3. Colisiones

8.4. Fantasmas

8.5. Control del juego

Diagrama A: Ciclo Principal

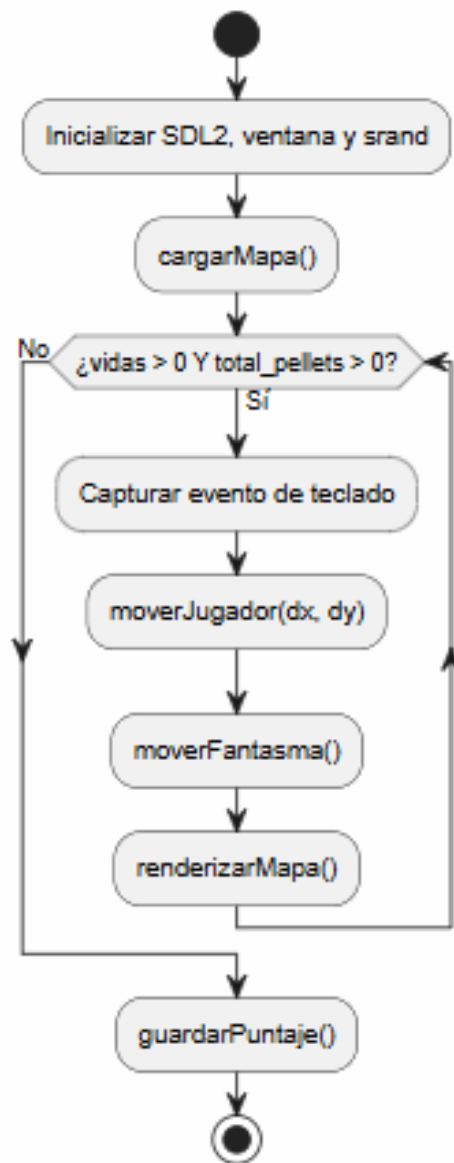


Figura 1: Diagrama de flujo del ciclo principal.

Diagrama B: Movimiento

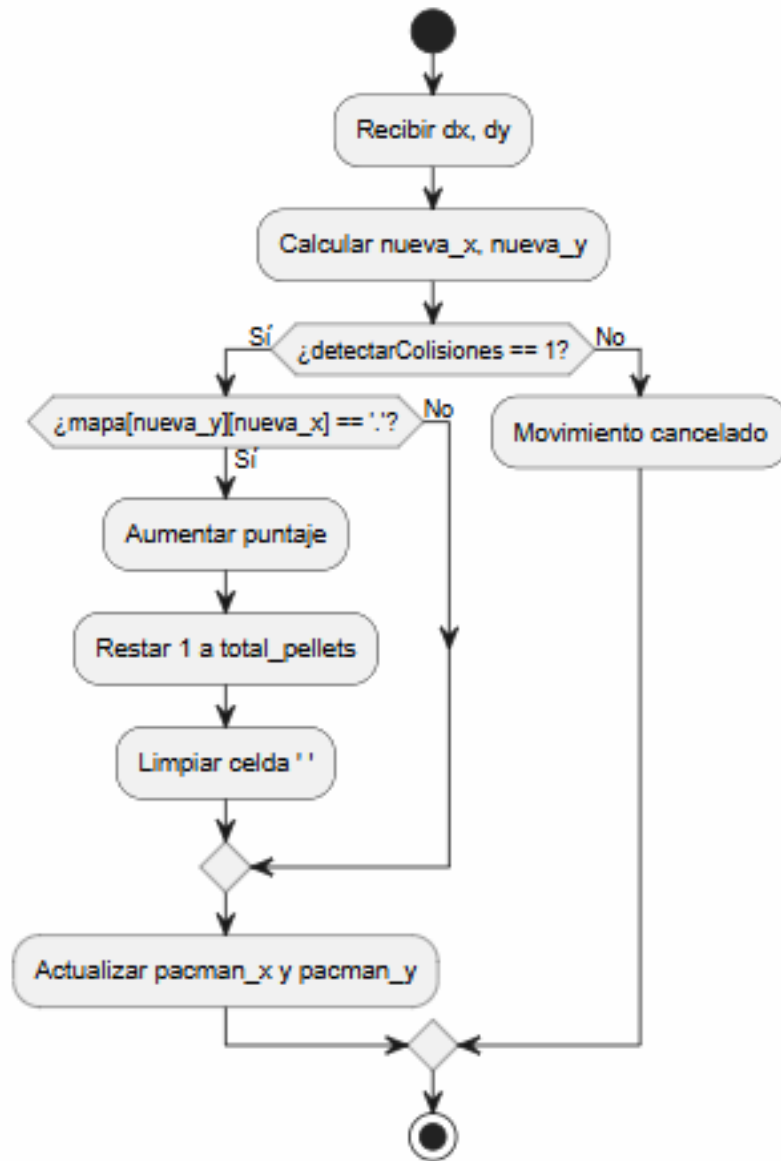


Figura 2: Diagrama de flujo del movimiento del jugador.

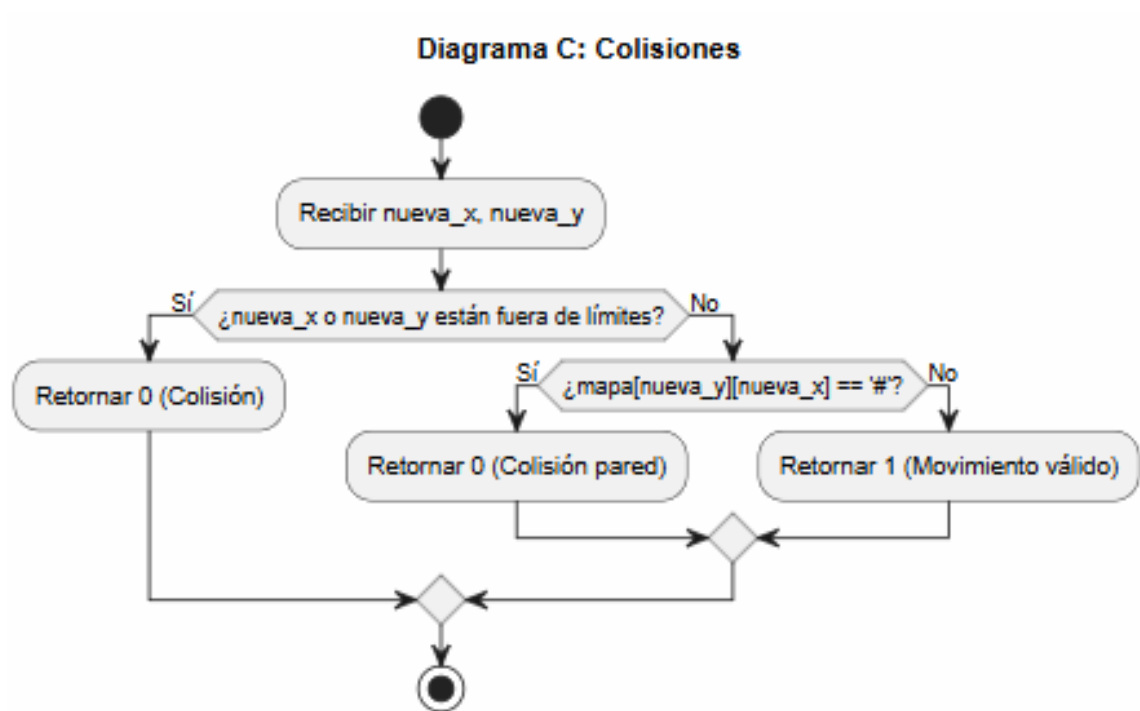


Figura 3: Diagrama de flujo de la detección de colisiones.

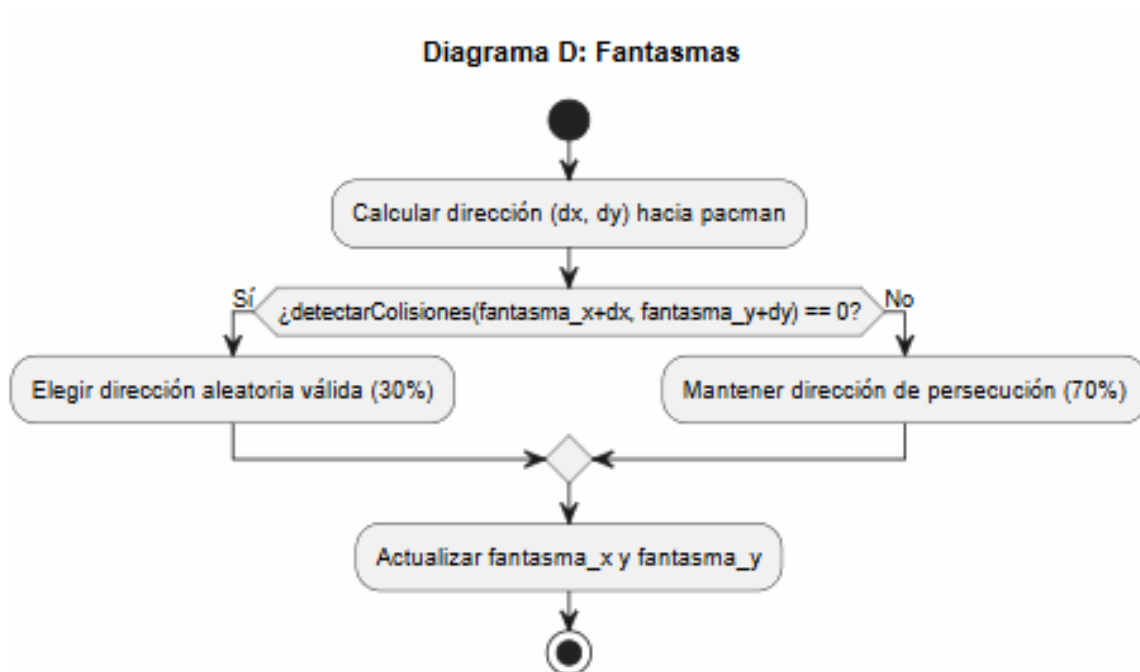


Figura 4: Diagrama de flujo del movimiento de fantasmas.

Diagrama E: Control del Juego

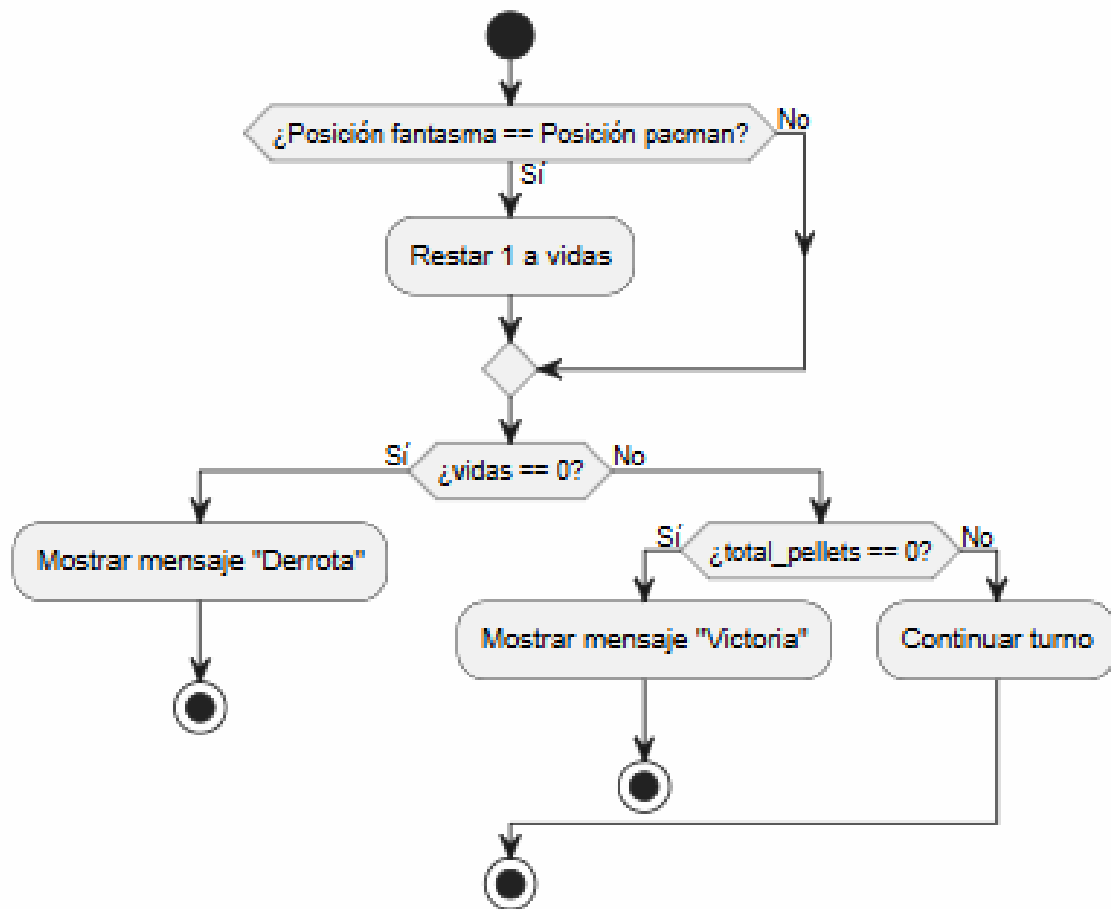


Figura 5: Diagrama de flujo del control general del juego.

9. Pseudocódigo

9.1. Módulo cargarMapa

```
función cargarMapa():
    ABRIR archivo "mapa.txt" en modo lectura
    SI el archivo NO existe ENTONCES
        IMPRIMIR "Error crítico: Falta archivo mapa.txt"
        TERMINAR ejecución
    FIN SI

    total_pellets = 0

    PARA f DESDE 0 HASTA FILAS - 1:
        PARA c DESDE 0 HASTA COLUMNAS - 1:
            LEER carácter del archivo
            SI el carácter es salto de línea ENTONCES
                repetir lectura para la misma columna
            FIN SI

            mapa[f][c] = carácter

            SI mapa[f][c] == 'P' ENTONCES
                pacman_x = c
                pacman_y = f
                mapa[f][c] = ' '
            FIN SI

            SI mapa[f][c] == 'F' ENTONCES
                fantasma_x = c
                fantasma_y = f
                mapa[f][c] = ' '
            FIN SI

            SI mapa[f][c] == '.' ENTONCES
                total_pellets = total_pellets + 1
            FIN SI
        FIN PARA
    FIN PARA

    CERRAR archivo
FIN función
```

9.2. Módulo detectarColisiones

```
función detectarColisiones(nueva_x, nueva_y) RETORNA ENTERO:
    SI nueva_x < 0 O nueva_x >= COLUMNAS O
        nueva_y < 0 O nueva_y >= FILAS ENTONCES
            RETORNAR 0
    FIN SI
```

```

    SI mapa[nueva_y][nueva_x] == '#' ENTONCES
        RETORNAR 0
    FIN SI

    RETORNAR 1
FIN FUNCIÓN

```

9.3. Módulo moverJugador

```

FUNCIÓN moverJugador(dx, dy):
    nueva_x = pacman_x + dx
    nueva_y = pacman_y + dy

    SI detectarColisiones(nueva_x, nueva_y) == 1 ENTONCES
        pacman_x = nueva_x
        pacman_y = nueva_y

        SI mapa[pacman_y][pacman_x] == '.' ENTONCES
            mapa[pacman_y][pacman_x] = ' '
            puntaje = puntaje + 10
            total_pellets = total_pellets - 1
        FIN SI
    FIN SI

    moverFantasma()
FIN FUNCIÓN

```

9.4. Módulo moverFantasma

```

FUNCIÓN moverFantasma():
    dx = 0
    dy = 0

    SI número_aleatorio < 70 ENTONCES
        SI pacman_x < fantasma_x ENTONCES dx = -1
        SI pacman_x > fantasma_x ENTONCES dx = 1
        SI pacman_y < fantasma_y ENTONCES dy = -1
        SI pacman_y > fantasma_y ENTONCES dy = 1
    SINO
        ELEGIR una dirección aleatoria
    FIN SI

    nueva_fx = fantasma_x + dx
    nueva_fy = fantasma_y + dy

    SI detectarColisiones(nueva_fx, nueva_fy) == 1 ENTONCES
        fantasma_x = nueva_fx
        fantasma_y = nueva_fy
    FIN SI

```

```

    SI fantasma_x == pacman_x Y fantasma_y == pacman_y ENTONCES
        vidas = vidas - 1
        reiniciar posiciones del jugador y fantasma
    FIN SI
FIN FUNCIÓN

```

9.5. Módulo guardarPuntaje

```

FUNCIÓN guardarPuntaje():
    ABRIR archivo "puntajes.txt" en modo añadir
    SI el archivo NO se puede abrir ENTONCES
        IMPRIMIR "Error al intentar escribir"
        RETORNAR
    FIN SI

    ESCRIBIR "Jugador_UNAM - Record Final: " + puntaje
    CERRAR archivo
    IMPRIMIR "Registro guardado con éxito"
FIN FUNCIÓN

```

10. Modularización

El programa se organiza en funciones para separar responsabilidades y facilitar la comprensión del código.

Función	Responsabilidad
cargarMapa()	Lee mapa.txt, llena la matriz, localiza jugador y fantasma, y cuenta pellets.
moverJugador(int dx, int dy)	Calcula el movimiento del jugador, valida colisiones y consume pellets.
moverFantasma()	Aplica reglas simples de movimiento y detecta choque con el jugador.
detectarColisiones(int nueva_x, int nueva_y)	Indica si una posición está dentro del mapa y no contiene pared.
renderizarMapa(SDL_Renderer* renderer)	Dibuja paredes, pellets, jugador y fantasma en la ventana SDL2.
guardarPuntaje()	Añade el puntaje final al archivo puntajes.txt.

11. Uso de herramientas de inteligencia artificial

Durante el desarrollo se permite utilizar herramientas de IA únicamente como apoyo. En este proyecto, su uso se considera asistencia para:

- Organizar la documentación del proyecto.
- Revisar la correspondencia entre requisitos y módulos del programa.
- Formular pseudocódigo claro a partir de las funciones implementadas.
- Proponer estructura para diagramas de flujo.
- Explicar errores o dudas relacionadas con SDL2 y programación en C.

El equipo debe comprender el código entregado, poder explicar cada módulo y modificarlo en caso de ser necesario. La IA no sustituye el desarrollo ni la comprensión del programa.

12. Bitácora de desarrollo

Pendiente. Esta sección se completará posteriormente con la información proporcionada por el equipo.

La bitácora deberá incluir:

- Avances realizados.
- Problemas encontrados.
- Decisiones de diseño.
- Distribución de tareas.
- Pruebas realizadas.
- Uso de herramientas de IA.

Para cada uso de IA se registrará la herramienta utilizada, la consulta realizada, un fragmento de la respuesta obtenida y una explicación de cómo se integró esa respuesta al proyecto.

13. Conclusiones

El proyecto cumple con el objetivo de integrar fundamentos de programación en una aplicación gráfica sencilla. La solución usa C, SDL2, arreglos bidimensionales, funciones, manejo de archivos, control de eventos, colisiones, puntaje, vidas y condiciones de victoria o derrota. La complejidad se mantiene en un nivel introductorio-intermedio, evitando inteligencia artificial avanzada y enfocándose en la correcta aplicación de programación estructurada.